

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1223	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL 2 DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	D. Srijana Bai	Reg.No.:	192571067
Department:	CS & BIOSCIENCE	Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I D. Sujana Bai (192571062) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)	
Q1	3	4	5	4	2	18
Q2	4	3	4	3	3	17
Q3	3	2	3	3	3	14
Q4	4	4	2	4	2	16
Q5	3	3	4	2	3	15

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem(4)	Identifies most errors with minor omissions(3)	Identifies limited errors; misses key issues(2)	Unable to identify errors or misunderstands the problem(1)
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification(4)	Applies suitable debugging methods with minor gaps(3)	Uses basic debugging methods with limited analysis(2)	No systematic debugging approach followed(0)
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output(5)	Provides working solution with minor errors(4)	Partially correct solution with limited functionality(3)	Incorrect solution or fails to resolve errors(0)
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality(4)	Performs optimization with noticeable improvements(3)	Limited optimization with minimal improvements(2)	No optimization applied or inefficient implementation(0)
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation(3)	Provides adequate testing and documentation with minor issues(2)	Limited testing and incomplete documentation(1)	No proper testing or documentation provided(0)

1.A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

A). Identification of Errors / Bugs

The processor performs signed 8-bit addition using 2's complement representation.

Given:

+120 and +50

Convert to binary:

- 120 = 01111000
- 50 = 00110010

Perform addition:

10101010

The result obtained is 10101010₂.

Since the MSB is 1, the processor interprets the result as a negative number.

This is incorrect because:

$$120 + 50 = 170$$

The range of an 8-bit signed integer is:

-128 to +127

Since 170 exceeds +127, overflow has occurred.

Debugging Approach & Analysis

Overflow in 2's complement addition occurs when:

- Two positive numbers produce a negative result.
- Two negative numbers produce a positive result.

In this example:

Operand 1 sign bit = 0

Operand 2 sign bit = 0

Result sign bit = 1

Therefore, overflow is detected.

Another method:

Overflow = Carry into MSB XOR Carry out of MSB

If the two carries differ, overflow has occurred.

Correctness of Debugged Solution

The processor should not treat 10101010₂ as a valid answer.

Instead:

- Actual mathematical result = 170
- Since 170 cannot be represented in signed 8-bit format, an overflow flag must be raised.

Optimization Techniques Applied

1. Add an Overflow Detection Circuit.
2. Monitor sign bits during arithmetic operations.
3. Use wider registers (16-bit or 32-bit) when larger values are expected.
4. Generate exception signals whenever overflow occurs.

Testing and Documentation

Test Case 1:

$$60 + 40 = 100$$

No Overflow

Test Case 2:

$$120 + 50 = 170$$

Overflow Detected

Test Case 3:

$$-100 + (-50)$$

Overflow Detected

Final Result

The error occurs because the sum exceeds the valid range of signed 8-bit representation.

Overflow Flag = 1

Correct Action = Detect overflow and notify the processor.

2.A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

A).Identification of Errors / Bugs

- A ripple carry adder shows excessive delay during arithmetic operations.
- The issue occurs because each carry must propagate through every bit position before the final sum is produced.

Example:

$$A = 1111$$

$$B = 0001$$

1. Carry generated at bit 0 must travel through bits 1, 2, and 3.
2. This creates propagation delay.
3. Debugging Approach & Analysis

In a Ripple Carry Adder:

C1 depends on C0

C2 depends on C1

C3 depends on C2

C4 depends on C3

Therefore, carries are generated sequentially.

1. For large word sizes (16-bit, 32-bit, 64-bit), the delay becomes significant.
2. Correctness of Debugged Solution
3. Replace Ripple Carry Adder with Carry Look-Ahead Adder (CLA).

Generate Signal:

$$G_i = A_i \times B_i$$

Propagate Signal:

$$P_i = A_i + B_i$$

Carries are calculated simultaneously.

Example:

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1C_1$$

$$C_3 = G_2 + P_2C_2$$

$$C_4 = G_3 + P_3C_3$$

Thus carries are available immediately without waiting.

Optimization Techniques Applied

1. Use Carry Look-Ahead Adder.
2. Parallel carry generation.

3. Reduce critical path delay.
4. Improve processor throughput.

Testing and Documentation

Ripple Carry Adder:

- Delay increases with bit size.
- Carry Look-Ahead Adder:
- Carries computed simultaneously.
- Execution time reduced significantly.

Final Result

The bottleneck is carry propagation delay.

Replacing Ripple Carry Adder with CLA greatly improves speed and makes it suitable for real-time systems.

3.A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

A).Identification of Errors / Bugs

A Booth Multiplier produces incorrect results when multiplying negative numbers.

Possible issues:

1. Incorrect bit-pair checking.
2. Wrong arithmetic right shift.
3. Missing sign extension.
4. Incorrect handling of Q_0 and Q_{-1} .

Example:

$$-6 \times +5$$

Expected Result = -30

If any of the above steps are implemented incorrectly, the final product becomes wrong.

Debugging Approach & Analysis

Booth's Algorithm examines:

Q_0Q_{-1}

Rules:

00 → No operation

11 → No operation

01 → $A = A + M$

10 → $A = A - M$

Common Bug 1:

Using logical shift instead of arithmetic shift.

Logical shift inserts 0 into MSB.

This destroys the sign of negative numbers.

Common Bug 2:

Improper sign extension during shifting.

Negative numbers must preserve MSB.

Common Bug 3:

Incorrect update of Booth bit Q_{-1} .

This causes incorrect addition/subtraction decisions.

Correctness of Debugged Solution

Correct Procedure:

Step 1:

Check Q_0Q_{-1}

Step 2:

Perform addition or subtraction if required

Step 3:

Perform arithmetic right shift

Step 4:

Preserve sign bit

Step 5:

Update Q_{-1} correctly

Following these steps produces:

$$(-6) \times (+5)$$

$$= -30$$

Optimization Techniques Applied

1. Efficient bit-pair checking.
2. Proper arithmetic shift implementation.
3. Hardware sign-extension support.
4. Reduced addition/subtraction operations.

Testing and Documentation

Test Case 1:

$$(-6) \times (+5)$$

Expected = -30

Obtained = -30

Test Case 2:

$$(-4) \times (-3)$$

Expected = +12

Obtained = +12

Test Case 3:

$$(+7) \times (-2)$$

Expected = -14

Obtained = -14

Final Result

The main source of error is usually incorrect arithmetic shifting or sign extension.

After correction, Booth's algorithm correctly handles signed multiplication and produces accurate results.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

A). Identification of Errors / Bugs

An embedded system uses the Restoring Division Algorithm and experiences poor performance due to repeated restoration operations.

Given:

$$\text{Dividend} = 13 = 1101_2$$

$$\text{Divisor} = 3 = 0011_2$$

In restoring division:

1. Shift left the Accumulator (A) and Quotient (Q).

2. Subtract divisor from accumulator.
3. If result becomes negative, restore the previous value.
4. Set quotient bit accordingly.

The major inefficiency occurs whenever the accumulator becomes negative because an additional restoration step is required.

Debugging Approach & Analysis

Example:

$A = 0001$

$M = 0011$

$A - M$

1110 (Negative)

Since the result is negative, the restoring algorithm performs:

0001

This extra restoration operation increases:

- Execution time
- Hardware activity
- Power consumption

When division involves many bits, restoration may occur several times, reducing efficiency.

Correctness of Debugged Solution

The issue can be eliminated by replacing Restoring Division with Non-Restoring Division.

In Non-Restoring Division:

- Positive remainder $\rightarrow$ Subtract divisor.
- Negative remainder $\rightarrow$ Add divisor in the next cycle.
- Immediate restoration is not performed.

This removes unnecessary operations.

Example:

$13 \div 3$

Quotient = $0100_2 = 4$

Remainder = $0001_2 = 1$

The final answer remains correct while requiring fewer operations.

Optimization Techniques Applied

1. Replace restoring division with non-restoring division.
2. Eliminate restoration after every negative remainder.
3. Reduce arithmetic operations.
4. Improve processor efficiency and speed.
5. Lower hardware complexity.

Testing and Documentation

Test Case 1:

$13 \div 3$

Quotient = 4

Remainder = 1

Test Case 2:

$15 \div 2$

Quotient = 7

Remainder = 1

Test Case 3:

$20 \div 5$

Quotient = 4

Remainder = 0

Results from Non-Restoring Division match the expected outputs while executing faster.

Final Result

- ✓ The performance bottleneck is caused by repeated restoration operations.
- ✓ Using Non-Restoring Division eliminates unnecessary restoration steps and improves execution speed, making it preferable in modern processor

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

A).Identification of Errors / Bugs

A scientific application using IEEE 754 single-precision floating-point representation produces inaccurate results when adding very large and very small numbers.

Example:

100000000 + 1

In single precision:

100000000 is represented with a large exponent.

1 is represented with a much smaller exponent.

During addition, the smaller number may be lost due to limited mantissa precision.

Result obtained:

100000000 instead of 100000001

This phenomenon is called Loss of Significance.

Debugging Approach & Analysis

IEEE 754 addition involves:

Step 1: Exponent Alignment

Before addition, exponents must be equal.

Large Number:

$$1.011111 \times 2^{26}$$

Small Number:

$$1.000000 \times 2^0$$

To align exponents, the mantissa of the smaller number is shifted right many times.

After repeated shifting:

0.000000000000000000000001

The value becomes too small to be represented accurately.

Step 2: Mantissa Addition

The shifted value contributes very little to the final result.

Step 3: Normalization

The result is normalized to IEEE 754 format.

Step 4: Rounding

Excess bits are discarded according to IEEE 754 rounding rules.

This introduces additional error.

Root Cause of the Problem

1. Limited mantissa size in single precision.
2. Exponent alignment causes loss of small values.
3. Rounding introduces precision errors.

4. Repeated calculations accumulate errors.

Correctness of Debugged Solution

Use Double Precision instead of Single Precision.

Single Precision:

- 32 bits
- 23-bit mantissa

Double Precision:

- 64 bits
- 52-bit mantissa

Double precision stores significantly more fractional information and reduces rounding errors.

Optimization Techniques Applied

1. Double Precision Arithmetic

Improves numerical accuracy.

2. Kahan Summation Algorithm

Reduces rounding errors during repeated additions.

3. Reorder Computations

Add smaller numbers together first before adding larger values.

4. Avoid Unnecessary Conversions

Prevent repeated precision loss.

5. Use Scientific Libraries

Optimized numerical libraries provide more stable computations.

Testing and Documentation

Test Case 1:

$1.5 + 2.25$

Expected = 3.75

Obtained = 3.75

Test Case 2:

$100000000 + 1$

Single Precision:

100000000

Double Precision:

100000001

Test Case 3:

$0.1 + 0.2$

Expected ≈ 0.3

Small rounding difference observed due to binary representation.

Final Result

- ✓ The inaccuracy occurs because exponent alignment and rounding cause loss of precision in single-precision floating-point arithmetic.
- ✓ Using double precision and improved numerical algorithms significantly reduces errors and provides more reliable scientific computations.